

Capgemini DevOps & Cloud Interview

Structured Q&A Guide | Prepared by Sanket Ajay Chopade

This guide covers 17 real interview questions asked at Capgemini, answered with structured points, relevant tooling context, and internship-anchored insights. Use it to rehearse answers out loud, not just read them.

Section 1: CI/CD Pipeline

Q1. Can you explain the CI/CD pipeline process from code commit to production deployment?

Pipeline Stages

- **Code Commit** — Developer pushes to a feature branch; pull/merge request triggers the pipeline.
- **Build** — Source is compiled/packaged; Docker image built and tagged (e.g., with Git SHA).
- **Static Analysis & Secrets Scan** — SonarQube checks code quality; Trivy scans image for CVEs.
- **Unit & Integration Tests** — Automated test suites run; failures block the pipeline immediately.
- **Artifact Push** — Verified image pushed to Amazon ECR with an immutable tag.
- **Deploy to Staging** — Helm chart updated via Argo CD; synced to staging Kubernetes namespace.
- **Smoke / E2E Tests** — Lightweight health checks and end-to-end tests run against staging.
- **Approval Gate** — Manual approval (GitHub Actions environment protection or Argo CD sync policy).
- **Production Deploy** — Argo CD syncs the prod namespace; Kubernetes performs rolling update.
- **Post-Deploy Monitoring** — Prometheus/Grafana alerts active; rollback automated on error-rate breach.

Tip: Anchor this in your Hisan Labs experience: Jenkins + GitHub Actions for CI, Argo CD for CD, ECR as registry.

Q2. What are the common failure points in a CI/CD pipeline, and how do you address them?

Failure Points & Mitigations

- **Flaky Tests** — Non-deterministic tests create false failures. Fix: quarantine flaky tests, use retry logic sparingly, fix root causes.
- **Secrets Leakage** — Hardcoded credentials in code/logs. Fix: use AWS Secrets Manager / External Secrets Operator; mask secrets in CI logs.
- **Slow Pipelines** — Long build times block developer velocity. Fix: parallelize stages, cache dependencies (e.g., Docker layer cache, npm cache).
- **Environment Drift** — Staging differs from production. Fix: Terraform for infra parity; Helm values files per environment.
- **Image Tag Ambiguity** — Using 'latest' tag causes unpredictable deploys. Fix: always tag with Git SHA + semantic version.
- **Missing Rollback Plan** — No fast path back on failure. Fix: Argo CD automated rollback on health check failure; Helm rollback command.

- **Approval Bottlenecks** — Manual gates become blockers. Fix: automate lower-env approvals; keep manual gate only for production.

Section 2: Terraform & Infrastructure as Code

Q3. How do you manage multi-cloud infrastructure using Terraform?

Key Practices

- **Provider Blocks** — Declare separate provider blocks per cloud (aws, azure, google) with aliased configurations.
- **Modular Structure** — Create reusable modules per resource type (network, compute, database); each module works across providers via input variables.
- **Workspace Isolation** — Use Terraform workspaces or separate state files per cloud/environment to avoid cross-cloud blast radius.
- **Variable Abstraction** — Abstract cloud-specific differences behind standardized variable interfaces in modules.
- **Remote Backend Per Cloud** — Store state in S3 (AWS), GCS (GCP), or Azure Blob per cloud to keep state isolated.
- **CI/CD Integration** — Run terraform plan in PR checks; terraform apply only after approval; use Atlantis or Terraform Cloud for governance.

Tip: Be honest: at Hisan Labs you worked primarily on AWS. Frame multi-cloud as your architectural understanding, not hands-on experience you don't have.

Q4. How do you handle Terraform state management in multi-cloud projects?

State Management Strategy

- **Remote Backend** — Never use local state in production. Use S3 + DynamoDB locking for AWS; equivalent per cloud.
- **State Isolation** — Separate state files per environment (dev/staging/prod) and per cloud to limit the blast radius of state corruption.
- **State Locking** — DynamoDB table for AWS backend prevents concurrent applies from corrupting state.
- **State Encryption** — Enable SSE on the S3 bucket; restrict access via IAM policies — only CI/CD role should write state.
- **Sensitive Output Handling** — Mark outputs containing secrets as `sensitive = true` so they don't appear in logs.
- **State Backup** — Enable S3 versioning on the state bucket; allows point-in-time recovery if state is corrupted.
- **Import vs. Re-create** — Use `terraform import` to bring existing resources under management rather than destroying and recreating them.

Q10. How do you handle Terraform state drift caused by manual changes?

Detection & Remediation

- **Detect Drift** — Run `terraform plan` regularly (scheduled in CI/CD) to surface differences between state and real infrastructure.

- **Drift Alerting** — Use Terraform Cloud drift detection or a scheduled pipeline job that fails on non-empty plan output.
- **Evaluate Before Acting** — Understand *why* the manual change was made. Sometimes it was a legitimate emergency fix.
- **Option A — Reconcile**: Codify the manual change in Terraform, then apply to bring state in sync.
- **Option B — Revert**: If the manual change was wrong, *terraform apply* to restore the desired state.
- **Prevent Recurrence** — Restrict console/CLI access via IAM SCPs; enforce infrastructure-change-only-via-Terraform policy.
- **Use terraform refresh** — Refreshes state file with real infra values without making changes (useful for investigation).

Q11. How do you make Terraform code reusable?

Reusability Patterns

- **Modules** — Extract repeated resource groups into modules with input variables and outputs. Example: a VPC module used across dev/staging/prod.
- **Variable Defaults** — Provide sensible defaults so modules work out-of-the-box but remain overridable.
- **Output Exposure** — Export IDs, ARNs, and endpoints as outputs so dependent modules can reference them without hardcoding.
- **Terraform Registry / Private Registry** — Publish internal modules to a private registry for team-wide reuse.
- **Versioning** — Tag module releases; reference specific versions in consuming configurations to avoid breaking changes.
- **DRY via locals and for_each** — Use *locals* to avoid repeating expressions; *for_each* to create multiple similar resources from a map.

Section 3: Kubernetes

Q5. What does 'large-scale Kubernetes' mean in your experience, and how do you manage it?

Be direct: as a fresher, your EKS experience was internship-scale. Frame what you know architecturally.

What Large-Scale Means

- Dozens of namespaces, hundreds of pods, multiple node groups, multi-team access control.
- High pod churn, frequent rolling deployments, autoscaling under real load.

How to Manage It

- **Namespace Isolation** — Separate teams/environments into namespaces with RBAC and NetworkPolicies.
- **Resource Requests & Limits** — Every pod must declare CPU/memory requests and limits to enable proper scheduling.
- **HPA & KEDA** — Horizontal Pod Autoscaler scales pods on CPU/memory; KEDA enables event-driven scaling (SQS queue depth, etc.).
- **Cluster Autoscaler / Karpenter** — Node-level scaling; Karpenter is faster and more cost-efficient on AWS.
- **GitOps with Argo CD** — All deployments driven from Git; no kubectl apply in production.
- **Observability** — Prometheus + Grafana for metrics; structured logging with CloudWatch or Loki.

Q6. How do you troubleshoot a failing Pod in Kubernetes?

Systematic Troubleshooting Steps

- **Step 1 — Check Pod status:** `kubectl get pod <name> -n <ns>` — look at STATUS and RESTARTS count.
- **Step 2 — Describe the Pod:** `kubectl describe pod <name>` — check Events section for scheduling failures, image pull errors, OOMKill.
- **Step 3 — Check logs:** `kubectl logs <name> --previous` for last container run; add `-c <container>` for multi-container pods.
- **Step 4 — Exec into container** (if running): `kubectl exec -it <name> -- /bin/sh` to inspect filesystem and env vars.
- **Step 5 — Check resource constraints:** Is the node under memory pressure? `kubectl top pod / kubectl top node`.
- **Step 6 — Check image:** Is the ECR image tag correct? Does the service account have pull permissions (IRSA)?
- **Step 7 — Check ConfigMaps/Secrets:** Missing or incorrectly mounted config causes CrashLoopBackOff.
- **Step 8 — Check NetworkPolicy:** Overly restrictive policies can block inter-pod communication.

Common Root Causes

- ImagePullBackOff — wrong tag, private registry auth missing
- CrashLoopBackOff — application error on startup, missing env var
- OOMKilled — memory limit too low
- Pending — insufficient cluster resources or node selector mismatch

Q7. Can you explain Blue-Green deployment in Kubernetes?

Concept

Blue-Green maintains two identical environments: Blue (current production) and Green (new version). Traffic switches atomically from Blue to Green, minimizing downtime and enabling instant rollback.

Implementation in Kubernetes

- **Two Deployments** — Deploy `app-blue` (v1) and `app-green` (v2) with identical replica counts.
- **Label-Based Routing** — Service selector points to `version: blue`. Switch to green by patching selector to `version: green`.
- **Zero-Downtime Switch** — The Service selector patch is atomic; traffic flips instantly with no pod restarts.
- **Rollback** — Patch selector back to `version: blue`. Old deployment still running — rollback is instant.
- **With Ingress / Argo Rollouts** — Argo Rollouts supports blue-green natively with traffic weight control and automated analysis.

Trade-offs

- Requires 2x resource capacity during the switch window
- Database schema changes must be backward-compatible during transition
- Simpler and safer than canary for stateless services

Q13. How does Kubernetes handle traffic spikes?

Mechanisms

- **Horizontal Pod Autoscaler (HPA)** — Scales pod count based on CPU/memory utilization or custom metrics (via metrics-server or Prometheus adapter).
- **KEDA** — Event-driven autoscaler that scales on external signals: SQS depth, Kafka lag, HTTP request rate.
- **Cluster Autoscaler / Karpenter** — When HPA needs more pods but nodes are full, CA/Karpenter provisions new EC2 nodes.
- **Pod Disruption Budgets** — Ensure minimum available replicas during node scale-down events.
- **Resource Requests** — Accurate requests enable the scheduler to bin-pack pods efficiently across nodes.
- **Readiness Probes** — Pods only receive traffic once they are truly ready, preventing traffic routing to cold-starting pods.

Q14. How do you perform rollbacks in Kubernetes deployments?

Rollback Methods

- **kubectl rollout undo** — Rolls back to the previous ReplicaSet revision. *kubectl rollout undo deployment/<name>*
- **Revision-specific rollback** — *kubectl rollout undo deployment/<name> --to-revision=3* targets a specific history entry.
- **Helm rollback** — *helm rollback <release> <revision>* reverts both the Deployment and associated ConfigMaps/Secrets.
- **Argo CD rollback** — In GitOps, rollback = reverting the Git commit. Argo CD syncs the previous Git state to the cluster.
- **Automated rollback** — Argo Rollouts can auto-rollback based on error rate or latency thresholds from Prometheus analysis.

Tip: Always investigate root cause before rollback — rolling back without understanding why fails again in the next deploy.

Section 4: Docker

Q8. How do you ensure Docker image consistency across environments?

Consistency Practices

- **Immutable Tags** — Tag images with Git SHA (e.g., *app:abc1234*), not 'latest'. Same tag = exact same image in every environment.
- **Multi-stage Builds** — Separate build and runtime stages; deterministic output regardless of host machine.
- **Pin Base Image Digest** — Reference base images by digest (*FROM node:20@sha256:...*) not floating tags.
- **Centralized Registry** — All environments pull from the same ECR repository. No local builds in staging/prod.
- **Build in CI Only** — Images are never built on developer machines or servers; only the CI pipeline builds and pushes.
- **.dockerignore** — Exclude dev tooling, node_modules, .env files from the build context for lean and consistent images.
- **Trivy Scan on Push** — Gate ECR push on a clean vulnerability scan; a failed scan never reaches any environment.

Section 5: SonarQube & Code Quality

Q9. Why did you integrate SonarQube into your CI/CD pipeline, and at what stage?

Why SonarQube

- Catches bugs, code smells, security hotspots, and code duplication automatically — without a human reviewer needing to spot them.
- Enforces a quality gate: pipeline fails if coverage drops below threshold or critical issues are introduced.
- Prevents technical debt from accumulating by surfacing issues at the point they are introduced.

Stage in the Pipeline

- **After unit tests, before Docker build.** This is the optimal position because:
 - Tests must pass first — no point analysing code that's already broken.
 - Fails before an image is built and pushed — cheaper than catching issues post-deploy.
 - Feedback loop is fast — developer gets quality report within minutes of the commit.

Quality Gate Enforcement

- Configure SonarQube Quality Gate to block merge if: coverage < 80%, critical vulnerabilities > 0, or duplication > 3%.

Section 6: Monitoring & Observability

Q12. What monitoring and observability tools have you worked with?

Tools & Their Roles

- **Prometheus** — Metrics collection and storage. Scrapes targets via pull model; stores time-series data. Used for Kubernetes resource and application metrics.
- **Grafana** — Visualization layer over Prometheus. Built dashboards for pod CPU/memory, deployment health, and request latency.
- **Datadog** — Full-stack observability (APM, logs, infrastructure, synthetic monitoring). Used for application-level tracing and log correlation at Hisan Labs.
- **CloudWatch** — AWS-native monitoring for EKS node metrics, ALB access logs, Lambda, and RDS. Container Insights for pod-level metrics.
- **Alertmanager** — Routes Prometheus alerts to Slack, PagerDuty, or email based on severity rules.

The Three Pillars of Observability

- **Metrics** — Quantitative, time-series data (Prometheus, CloudWatch)
- **Logs** — Textual event records (CloudWatch Logs, Datadog Logs)
- **Traces** — Request flow across services (Datadog APM, AWS X-Ray)

Section 7: AWS

Q15. How does AWS handle traffic spikes?

AWS Scaling Mechanisms

- **Auto Scaling Groups (ASG)** — Automatically add/remove EC2 instances based on CloudWatch alarms (CPU, network, custom metrics).

- **Application Load Balancer (ALB)** — Distributes traffic across healthy targets; scales horizontally automatically with no management overhead.
- **EKS + Karpenter** — Kubernetes workloads: HPA scales pods, Karpenter provisions right-sized EC2 nodes within seconds.
- **Lambda** — Serverless functions scale to thousands of concurrent executions automatically; ideal for burst traffic patterns.
- **CloudFront** — CDN caches static and dynamic content at edge; absorbs traffic spikes before they hit the origin.
- **RDS Read Replicas / Aurora Serverless** — Offloads read traffic; Aurora Serverless v2 scales database capacity in fractions of an ACU.
- **SQS as Buffer** — Decouple producers from consumers; queue absorbs traffic spikes and consumers process at their own pace.

Q16. What could cause errors when EC2 instances are running?

Common EC2 Runtime Errors

- **High CPU / Memory** — Application under load, memory leak, or runaway process. Check with *top*, *htop*, CloudWatch metrics.
- **Disk Full** — Application logs or temp files filling the root volume. Monitor with CloudWatch `disk_used_percent`; set log rotation.
- **Security Group / NACLs Misconfiguration** — Wrong inbound/outbound rules blocking expected traffic. Check with VPC Flow Logs.
- **IAM Permission Denied** — Instance role missing required S3/DynamoDB/Secrets Manager permissions. Check CloudTrail for AccessDenied events.
- **Application Crash** — Process exits due to unhandled exception. Use `systemd/journalctl` logs or application-level logging.
- **Network Connectivity** — Route table missing, NAT Gateway down, or VPC peering misconfiguration.
- **Corrupted AMI / Bad Patch** — OS update breaks a driver or dependency. Snapshot before patching; test in staging first.
- **EC2 Status Check Failure** — Instance-level or system-level check failure. May require stop/start (moves to different host) or volume recovery.

Section 8: Real-World Scenario

Q17. What is a real-time production issue you identified and how did you troubleshoot it?

Use the STAR format (Situation, Task, Action, Result). Below is a structured example anchored to real internship work — adapt it to your actual experience.

Situation

- During my internship at Hisan Labs, a microservice deployed to EKS started returning 5xx errors intermittently under moderate load. Prometheus alerted on error rate exceeding 5% on the `/api/orders` endpoint.

Task

- Identify whether the issue was application-level, infrastructure-level, or a configuration problem — and resolve it without taking the service down.

Action

- **1. Checked pod health:** `kubectl get pods` showed some pods in CrashLoopBackOff with high restart counts.
- **2. Inspected logs:** `kubectl logs --previous` revealed OOMKilled — pods were being terminated due to memory limit too low.
- **3. Checked Grafana:** Confirmed memory usage spiking to the limit (256Mi) under load while CPU was fine.
- **4. Root cause:** A recent code change introduced an in-memory cache with no eviction policy — cache grew unboundedly.
- **5. Immediate fix:** Updated Helm values to increase memory limit to 512Mi and applied; pods stabilized.
- **6. Permanent fix:** Developer added LRU eviction to the cache; limit brought back to 256Mi after confirming stability.

Result

- Error rate dropped to 0% within 3 minutes of the limit increase. No data loss. Added memory utilization alert at 80% threshold to catch this pattern earlier.

Tip: Always have a real story ready. If this scenario doesn't match yours, replace it with an actual incident from your internship — specificity beats generic answers every time.